Laboratorio: Heap y Priority Queue (Top-K)

ICC708-1: PROGRAMACIÓN AVANZADA

Nombre: Bárbara Constanza Sanhueza Bombín.

Carrera: Ingeniería informática.

Fecha: 15-04-2026.

Docente: Dr. Ricardo Gacitúa

Introducción

Dentro de este laboratorio se utilizó el módulo heapq de Python para el estudio del uso de estructuras de datos de tipo Heap y la aplicación de los mismos a problemas típicos de la obtención de los elementos Top-K.

Se implementaron soluciones mediante heaps y se compararon con soluciones tradicionales de tipo sorted() de modo que se pudiera evaluar la eficiencia de ambas soluciones mediante el tiempo de ejecución, intentando al final identificar en qué situaciones el uso de heaps resulta más ventajoso que otros métodos.

heapq vs sorted()

El módulo heapq implementa un min-heap, lo que permite acceder de forma eficiente al elemento más pequeño O(1) y realizar inserciones o extracciones en O(logn)O(\log n)O(logn). Esto lo hace ideal para problemas donde solo se necesitan los k mejores elementos (Top-k), sin necesidad de ordenar la lista.

Por otro lado, sorted() ordena completamente la lista con una complejidad de O(nlogn), lo cual puede ser menos eficiente cuando solo se requiere una pequeña parte de los datos.

El heap es más eficiente cuando k es mucho menor que n y sorted() puede ser más rápido para listas pequeñas cuando se requiere orden completo.
Tabla de tiempo de ejecución en segundos.

Item	Heapq	sorted()
problema 1	0.000038	0.000033
problema 2	0.000105	0.000069
ejercicio 2.1	0.000625	0.000043
ejercicio 2.2	0.000052	0.000577
ejercicio 3	0.000062	0.000044

Conclusión

Los resultados evidencian que heapq no siempre se comporta mejor que sorted(), sobre todo en arreglos pequeños.

Por otro lado, en problemas como la mezcla de listas ordenadas (Ejercicio 2.2), heapq muestra una ventaja muy importante, ya que al aplicarse sobre listas que ya están ordenadas, no tiene que ordenar desde cero todos los elementos. Esto nos dice que los heaps pueden ser especialmente útiles cuando se trabaja con datos que se están desarrollando de forma incremental o cuando solo se necesita una parte del conjunto total. Como conclusión, la elección entre heapq y sorted() depende del problema a tratar: para Top-K y volúmenes de datos grandes, el heap es más efectivo; sin embargo, para ordenar todos los elementos o conjuntos pequeños, sorted() sigue siendo una forma sencilla y eficaz.